

```
1 interface arb_if #(parameter N = 4) (input bit clk);
2
3     logic [N-1:0] req;
4     logic [N-1:0] gnt;
5     bit rst;
6
7
8     clocking cb @(posedge clk);
9     default input #2ns output negedge;
10    output /* #3ns */ req;
11    input /* #25ns */ gnt;
12 endclocking
13
14
15 `ifdef CLOCKING_BLOCK
16     modport TEST (
17         clocking cb,
18         output rst,
19         input clk
20     );
21 `else
22     modport TEST (
23         output req,rst,
24         input gnt,clk
25     );
26 `endif
27
28
29
30
31
32
33
34     modport MON(
35         clocking cb
36     );
37
38 endinterface
```

```

1 module arbiter #(parameter N = 4)(
2     arb_if.DUT arb_bus
3 );
4
5     logic [$clog2(N)-1:0] ptr;
6     int index;
7
8     always_ff @(posedge arb_bus.clk or posedge arb_bus.rst) begin
9         if (arb_bus.rst) begin
10             ptr <= 0;
11             arb_bus.gnt <= 0;
12         end else begin
13             arb_bus.gnt <= 0;
14             for (int i = 0; i < N; i++) begin
15                 index = (ptr + i) % N;
16                 if (arb_bus.req[index]) begin
17                     arb_bus.gnt[index] <= 1;
18                     ptr <= index + 1;
19                     break;
20                 end
21             end
22         end
23     end
24
25 endmodule

```

```

1 program test #(parameter N = 4)(
2     arb_if.TEST arb_bus
3 );
4
5 logic [N-1:0] req;
6 logic [N-1:0] gnt;
7 bit rst;
8
9 assign clk = arb_bus.clk;
10 assign arb_bus.rst = rst;
11
12
13 initial begin
14     `ifdef CLOCKING_BLOCK
15         // Initialize
16         rst = 1;
17         arb_bus.cb.req <= 0;
18         #20 rst = 0;
19         $display("=== Test 1: Single request ===");
20         @(arb_bus.cb);
21         arb_bus.cb.req <= 4'b0001;
22         repeat(3) @(arb_bus.cb);
23         check_grant(4'b0001);
24
25         $display("=== Test 2: Round-robin priority ===");
26         arb_bus.cb.req <= 4'b1111;
27         repeat(8) begin
28             @(arb_bus.cb);
29             check_one_hot(arb_bus.cb.gnt);
30             $display("Grant = %4b", arb_bus.cb.gnt);
31         end
32     `else
33         rst = 1;
34         #10 rst = 0;
35         arb_bus.req = 4'b0001;
36         repeat(5) @(posedge clk);
37         arb_bus.req = 4'b1010;
38         repeat(5) @(posedge clk);
39         arb_bus.req = 4'b0000;
40         repeat(3) @(posedge clk);
41     `endif
42     $stop;
43 end
44
45
46 task check_one_hot(logic [3:0] gnt);
47     if ($countones(gnt) != 1)
48         $error("Invalid grant: %4b (not one-hot)", gnt);
49 endtask
50
51
52 task check_grant(logic [3:0] expected);
53     `ifdef CLOCKING_BLOCK
54         if (arb_bus.cb.gnt != expected)
55             $error("Grant mismatch: Expected %4b, Got %4b", expected, arb_bus.cb.gnt);
56         else
57             $display("PASS: Grant = %4b", arb_bus.cb.gnt);
58     `else
59         if (arb_bus.gnt != expected)
60             $error("Grant mismatch: Expected %4b, Got %4b", expected, arb_bus.gnt);
61         else

```

```
1 module monitor(  
2   arb_if.MON arb_bus  
3 );  
4  
5  
6 initial begin  
7   $monitor("[%0t] REQ = %b | GNT = %b", $time, arb_bus.req, arb_bus.gnt);  
8 end  
9  
10  
11 endmodule
```

```
1 `include "interface.sv"
2 `include "test.sv"
3 `include "monitor.sv"
4
5 module top;
6
7     parameter N = 4;
8     bit clk;
9
10    initial begin
11        clk = 0;
12        forever begin
13            #1ns clk = ~clk;
14        end
15    end
16
17    arb_if #(N) arb_bus (clk);
18    arbiter #(N) arb_dut (arb_bus.DUT);
19    test      #(N) tb      (arb_bus.TEST);
20    monitor          mon      (arb_bus.MON);
21
22 endmodule
```

```
1 vlog design.sv interface.sv monitor.sv test.sv testbench.sv +define+CLOCKING_BLOCK
2
3 vsim work.top
4
5 coverage save -onexit cov.ucdb
6
7 run -all
```